

Universidad Nacional del Sur

Departamento de Matemática

Métodos para el Análisis de Datos I

Código 8142 | Licenciatura en Matemática Aplicada

Unidad 4

Aprendizaje Supervisado

Dr. José Bavio

2026

Índice

1	Introducción al Aprendizaje Supervisado	3
1.1	El paradigma de generalización	3
1.2	Bias-variance tradeoff	3
1.3	Partición de datos y validación cruzada	4
2	Árboles de Decisión	4
2.1	Concepto y estructura	4
2.2	Criterios de partición	5
2.2.1	Índice de Gini	5
2.2.2	Entropía (ganancia de información)	5
2.3	Construcción: algoritmo CART	5
2.4	Poda	6
2.5	Ventajas y limitaciones	6
3	Random Forest	6
3.1	Bagging	6
3.2	Random Forest: decorrelación de árboles	6
3.3	Estimación out-of-bag (OOB)	7
3.4	Importancia de variables	7
3.4.1	Importancia por impureza (MDI)	7
3.4.2	Importancia por permutación (MDA)	7
3.5	Hiperparámetros principales	8
4	Support Vector Machines (SVM)	8
4.1	Clasificador de margen máximo	8
4.2	Problema de optimización (caso separable)	8
4.3	SVM de margen blando (soft margin)	8
4.4	Truco del kernel	9
4.5	Extensión a clasificación multiclase	9
5	K-Nearest Neighbors (KNN)	9
5.1	Definición	9
5.2	Elección de k	10
5.3	La maldición de la dimensionalidad	10
6	Redes Neuronales Artificiales	10
6.1	La neurona artificial (perceptrón)	10
6.2	Perceptrón Multicapa (MLP)	10
6.3	Funciones de activación	11
6.4	Función de pérdida	11
6.5	Entrenamiento: descenso por gradiente y backpropagation	11
6.6	Variantes de SGD	11

6.7	Regularización en redes neuronales	12
7	Métricas de Evaluación para Clasificación	12
7.1	Matriz de confusión	12
7.2	Métricas derivadas	12
7.3	Curva ROC y AUC	13
7.4	Curva Precisión-Recall (PR)	13
7.5	Extensión a multiclase	13
8	Clases Desbalanceadas	13
8.1	Estrategias de resampling	14
8.1.1	Oversampling: SMOTE	14
8.1.2	Undersampling	14
8.2	Ajuste de pesos de clase	14
9	Ingeniería de Features	14
9.1	Transformaciones habituales	14
9.2	Selección de features	15
10	Comparación de Modelos: Grid Search y Pipelines	15
10.1	Búsqueda de hiperparámetros	15
10.2	Pipelines	15
	Resumen de la Unidad	16
	Referencias	16

1 Introducción al Aprendizaje Supervisado

El **aprendizaje automático** (*machine learning*, ML) es una rama de la inteligencia artificial que estudia algoritmos capaces de aprender patrones a partir de datos sin ser programados explícitamente para cada tarea.

Dentro del ML se distinguen tres grandes paradigmas:

- **Aprendizaje supervisado:** el modelo aprende a partir de pares entrada-salida etiquetados (x_i, y_i) .
- **Aprendizaje no supervisado:** no hay etiquetas; el objetivo es descubrir estructura en los datos (Unidad 6).
- **Aprendizaje por refuerzo:** un agente aprende tomando decisiones y recibiendo recompensas.

Definición 1.1 (Aprendizaje Supervisado). Dado un conjunto de entrenamiento $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$ donde $\mathbf{x}_i \in \mathbb{R}^p$ es el vector de *features* e y_i es la variable respuesta, el objetivo es aprender una función $f: \mathbb{R}^p \rightarrow \mathcal{Y}$ que generalice bien a datos nuevos.

Según el tipo de variable respuesta:

- Si $y_i \in \mathbb{R} \Rightarrow$ **regresión** (cubierta en Unidad 3).
- Si $y_i \in \{1, \dots, K\} \Rightarrow$ **clasificación** (tema central de esta unidad).

1.1 El paradigma de generalización

Aprender de los datos de entrenamiento no es el objetivo final: lo que importa es que el modelo *generalice*, es decir, que funcione bien sobre datos que no vio durante el entrenamiento.

El **error de generalización** (o error de prueba) es la pérdida esperada sobre nuevas observaciones:

$$\text{Err} = \mathbb{E}[\ell(y, f(\mathbf{x}))]$$

donde ℓ es la función de pérdida (por ejemplo, pérdida 0-1 para clasificación). El objetivo del aprendizaje supervisado es minimizar Err, no el error de entrenamiento.

1.2 Bias-variance tradeoff

Para modelos de regresión se puede demostrar que el error cuadrático medio de predicción descompone como:

$$\text{ECM}[f(\mathbf{x}_0)] = \underbrace{\text{Var}[\hat{f}(\mathbf{x}_0)]}_{\text{varianza}} + \underbrace{[\text{Bias}(\hat{f}(\mathbf{x}_0))]^2}_{\text{sesgo}^2} + \underbrace{\sigma_\varepsilon^2}_{\text{irreducible}}$$

La misma tensión existe en clasificación:

- Modelos **demasiado simples** tienen alto sesgo (*underfitting*).
- Modelos **demasiado complejos** tienen alta varianza (*overfitting*).

[Insertar curva de bias-variance vs. complejidad del modelo]

Figura 1: Compromiso sesgo-varianza en función de la complejidad del modelo.

1.3 Partición de datos y validación cruzada

Para estimar el error de generalización se divide el conjunto de datos:

- **Entrenamiento** (*training set*): ajuste de los parámetros del modelo.
- **Validación** (*validation set*): selección de hiperparámetros.
- **Prueba** (*test set*): evaluación final; se toca una *única vez*.

La **validación cruzada k -fold** divide el conjunto de entrenamiento en k partes (*folds*) iguales; en cada iteración se entrena con $k - 1$ folds y se evalúa en el restante. El error de validación cruzada es el promedio de los k errores:

$$CV_k = \frac{1}{k} \sum_{j=1}^k \text{Err}_j$$

Observación 1.1. La validación cruzada *estratificada* (stratified k -fold) preserva la proporción de clases en cada fold, lo que es fundamental con clases desbalanceadas.

2 Árboles de Decisión

2.1 Concepto y estructura

Un **árbol de decisión** es un modelo que particiona el espacio de features mediante reglas de la forma $x_j \leq t$ (*splits*) organizadas en una estructura jerárquica de árbol.

Definición 2.1 (Árbol de clasificación). Un árbol de clasificación T es una estructura formada por:

- **Nodo raíz:** contiene todo el conjunto de entrenamiento.

- **Nodos internos:** cada uno define un split $\{x_j \leq t\}$ que divide los datos en dos subconjuntos (subárboles izquierdo y derecho).
- **Hojas:** regiones terminales R_m que asignan una clase $\hat{y} = \arg \max_k \hat{p}_{mk}$, donde $\hat{p}_{mk}$ es la proporción de observaciones de clase k en la hoja m .

2.2 Criterios de partición

Para elegir el mejor split (j^*, t^*) se minimiza la impureza de los nodos hijos. Los criterios más usados son:

2.2.1 Índice de Gini

$$G(p) = \sum_{k=1}^K p_k(1 - p_k) = 1 - \sum_{k=1}^K p_k^2$$

donde p_k es la proporción de la clase k en el nodo. $G = 0$ indica nodo puro.

2.2.2 Entropía (ganancia de información)

$$H(p) = - \sum_{k=1}^K p_k \log_2(p_k)$$

La **ganancia de información** de un split es la reducción en entropía:

$$\Delta H = H(\text{padre}) - \frac{n_L}{n} H(\text{hijo}_L) - \frac{n_R}{n} H(\text{hijo}_R)$$

Observación 2.1. Gini y entropía producen resultados muy similares en la práctica. Gini es computacionalmente más eficiente al evitar el cálculo de logaritmos.

2.3 Construcción: algoritmo CART

El algoritmo **CART** (*Classification and Regression Trees*) construye el árbol de manera *greedy* (voraz): en cada nodo busca el split que maximiza la reducción de impureza, sin garantía de optimalidad global.

1. Iniciar con el nodo raíz conteniendo todos los datos.
2. Para cada feature j y umbral t , calcular la impureza ponderada del split.
3. Elegir el split (j^*, t^*) que minimiza la impureza total.
4. Repetir recursivamente en cada subárbol.
5. Detener cuando se cumple un criterio de parada (profundidad máxima, mínimo de muestras por hoja, etc.).

2.4 Poda

Árboles crecidos sin restricción tienen alta varianza. La **poda por complejidad de costo** (*cost-complexity pruning*) busca el subárbol $T \subset T_{\max}$ que minimiza:

$$C_{\alpha}(T) = \sum_{m=1}^{|T|} \sum_{i \in R_m} \mathbf{1}(y_i \neq \hat{y}_m) + \alpha|T|$$

donde $|T|$ es el número de hojas y $\alpha \geq 0$ es el parámetro de regularización (elegido por validación cruzada).

2.5 Ventajas y limitaciones

Ventajas	Limitaciones
Interpretabilidad directa	Alta varianza: pequeños cambios en los datos producen árboles muy distintos
Maneja features categóricos y numéricos	Fronteras de decisión solo axiales
No requiere normalización	Tendencia a sobreajuste si no se poda
Captura interacciones entre variables	Sesgo en features con muchas categorías

3 Random Forest

3.1 Bagging

El **bagging** (*bootstrap aggregating*, Breiman 1996) reduce la varianza promediando múltiples modelos inestables entrenados sobre muestras bootstrap del conjunto de entrenamiento.

Dado un conjunto $\mathcal{D}$ de n observaciones:

1. Para $b = 1, \dots, B$: extraer una muestra bootstrap $\mathcal{D}^{(b)}$ de tamaño n con reemplazo y entrenar un árbol $T^{(b)}$.
2. Predicción por mayoría de votos:

$$\hat{y}(\mathbf{x}) = \arg \max_k \sum_{b=1}^B \mathbf{1}(\hat{y}^{(b)}(\mathbf{x}) = k)$$

3.2 Random Forest: decorrelación de árboles

Definición 3.1 (Random Forest). **Random Forest** (Breiman, 2001) extiende el bagging añadiendo *aleatoriedad en la selección de features*: en cada split de cada

árbol, se elige el mejor split solo entre un subconjunto aleatorio de m features (en lugar de los p totales).

El parámetro m controla el grado de decorrelación:

- Valor por defecto para clasificación: $m = \lfloor \sqrt{p} \rfloor$
- Valor por defecto para regresión: $m = \lfloor p/3 \rfloor$

La decorrelación es esencial: si los árboles fueran muy correlacionados, el promedio no reduciría la varianza.

Proposición 3.1 (Reducción de varianza por promediado). *Sea ρ la correlación entre dos árboles cualesquiera y σ^2 la varianza de cada uno. La varianza del promedio de B árboles es:*

$$\rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$$

Cuando $B \rightarrow \infty$, la varianza tiende a $\rho\sigma^2$. Reducir ρ (mediante la aleatoriedad en features) es lo que diferencia RF del bagging puro.

3.3 Estimación out-of-bag (OOB)

Cada árbol $T^{(b)}$ se entrena sobre $\mathcal{D}^{(b)}$. Las observaciones *no incluidas* en $\mathcal{D}^{(b)}$ (aproximadamente 36.8% de $\mathcal{D}$) constituyen la muestra **out-of-bag** (OOB). El error OOB es un estimador casi insesgado del error de generalización y puede usarse en lugar de validación cruzada.

3.4 Importancia de variables

Random Forest provee dos medidas de importancia:

3.4.1 Importancia por impureza (MDI)

La importancia de la variable j es la reducción total de impureza (Gini o entropía) generada por los splits en j , promediada sobre todos los árboles:

$$\text{Imp}(j) = \frac{1}{B} \sum_{b=1}^B \sum_{\substack{t \in T^{(b)} \\ \text{split en } j}} \Delta G_t$$

3.4.2 Importancia por permutación (MDA)

Para cada árbol $T^{(b)}$ y cada variable j :

1. Calcular el error OOB base.
2. Permutar aleatoriamente los valores de x_j en el conjunto OOB.

3. Recalcular el error OOB con la permutación.
4. La importancia es el aumento promedio del error.

MDI sobreestima la importancia de variables con muchas categorías o alta cardinalidad. Para datos con features de muy distinta naturaleza, se recomienda la importancia por permutación (MDA).

3.5 Hiperparámetros principales

Hiperparámetro	Efecto	Valor típico
<i>n_estimators</i>	Número de árboles	100–500
<i>max_features</i>	Features considerados por split	$\sqrt{p}$ (clasif.)
<i>max_depth</i>	Profundidad máxima del árbol	None (sin límite)
<i>min_samples_leaf</i>	Mínimo de muestras por hoja	1–5
<i>bootstrap</i>	Usar muestras bootstrap	True

4 Support Vector Machines (SVM)

4.1 Clasificador de margen máximo

Para un problema de clasificación binaria con clases $y \in \{-1, +1\}$, se busca el hiperplano

$$\mathcal{H} = \{\mathbf{x} : \mathbf{w}^\top \mathbf{x} + b = 0\}$$

que separa las dos clases con el **máximo margen**. El margen es la distancia entre el hiperplano y las observaciones más cercanas de cada clase (*vectores de soporte*).

4.2 Problema de optimización (caso separable)

Definición 4.1 (SVM de margen duro). Minimizar $\frac{1}{2}\|\mathbf{w}\|^2$ sujeto a

$$y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 \quad \forall i = 1, \dots, n$$

El margen geométrico es $2/\|\mathbf{w}\|$, de modo que minimizar $\|\mathbf{w}\|^2$ equivale a maximizarlo.

4.3 SVM de margen blando (soft margin)

Cuando los datos no son linealmente separables, se introducen **variables de holgura** $\xi_i \geq 0$:

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2}\|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i \quad \text{sujeto a} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0$$

El parámetro $C > 0$ controla el **compromiso sesgo-varianza**:

- C grande: margen más pequeño, menos errores de entrenamiento, más riesgo de sobreajuste.
- C pequeño: margen más grande, mayor tolerancia a errores, más regularización.

4.4 Truco del kernel

La formulación dual del problema SVM depende de los datos solo a través de productos internos $\langle \mathbf{x}_i, \mathbf{x}_j \rangle$. Reemplazando el producto interno por una **función kernel** $K(\mathbf{x}_i, \mathbf{x}_j) = \langle \phi(\mathbf{x}_i), \phi(\mathbf{x}_j) \rangle$ se proyectan implícitamente los datos a un espacio de mayor dimensión sin calcularlo explícitamente.

Kernel	Fórmula	Parámetro(s)
Lineal	$K(\mathbf{x}, \mathbf{x}') = \mathbf{x}^\top \mathbf{x}'$	—
Polinomial	$K(\mathbf{x}, \mathbf{x}') = (\gamma \mathbf{x}^\top \mathbf{x}' + r)^d$	γ, r, d
RBF (Gaussiano)	$K(\mathbf{x}, \mathbf{x}') = \exp(-\gamma \ \mathbf{x} - \mathbf{x}'\ ^2)$	γ
Sigmoide	$K(\mathbf{x}, \mathbf{x}') = \tanh(\gamma \mathbf{x}^\top \mathbf{x}' + r)$	γ, r

El kernel RBF con γ pequeño produce fronteras suaves (mayor regularización); con γ grande produce fronteras muy ajustadas al entrenamiento (riesgo de sobreajuste).

4.5 Extensión a clasificación multiclase

Las SVM son binarias por definición. Las estrategias usuales para $K > 2$ clases son:

- **Uno contra todos (OvR)**: se entrena un clasificador por clase.
- **Uno contra uno (OvO)**: se entrena un clasificador para cada par de clases; la predicción final es por votación.

5 K-Nearest Neighbors (KNN)

5.1 Definición

Definición 5.1 (k -NN clasificador). Dado un punto de consulta $\mathbf{x}_0$, el clasificador k -NN asigna la clase mayoritaria entre sus k vecinos más cercanos en el conjunto de entrenamiento:

$$\hat{y}(\mathbf{x}_0) = \arg \max_c \sum_{i \in \mathcal{N}_k(\mathbf{x}_0)} \mathbf{1}(y_i = c)$$

donde $\mathcal{N}_k(\mathbf{x}_0)$ es el conjunto de los k índices con menor distancia a $\mathbf{x}_0$.

La distancia estándar es la euclídea; alternativas incluyen Manhattan, Minkowski (L^p) y distancia de Mahalanobis.

5.2 Elección de k

- $k = 1$: frontera de decisión muy irregular, alta varianza.
- k grande: frontera suave, alto sesgo.
- Se elige mediante validación cruzada; típicamente se prueban valores impares para evitar empates en clasificación binaria.

5.3 La maldición de la dimensionalidad

En espacios de alta dimensión ($p \gg 1$), las distancias entre puntos tienden a homogeneizarse: la diferencia entre el vecino más cercano y el más lejano se hace proporcionalmente pequeña. Esto deteriora drásticamente el rendimiento de k -NN.

k -NN **requiere normalización o estandarización** previa de los features, ya que es sensible a la escala de las variables.

6 Redes Neuronales Artificiales

6.1 La neurona artificial (perceptrón)

Una **neurona artificial** toma p entradas $x_1, \dots, x_p$, las combina linealmente y aplica una función de activación σ :

$$a = \sigma\left(\sum_{j=1}^p w_j x_j + b\right) = \sigma(\mathbf{w}^\top \mathbf{x} + b)$$

6.2 Perceptrón Multicapa (MLP)

Definición 6.1 (Perceptrón Multicapa). Un **MLP** (*Multilayer Perceptron*) es una red neuronal con capas totalmente conectadas (*fully connected*):

- Una **capa de entrada** de dimensión p .
- Una o más **capas ocultas** de neuronas con activación no lineal.
- Una **capa de salida** que produce la predicción final.

La salida de la capa l es:

$$\mathbf{h}^{(l)} = \sigma^{(l)}(\mathbf{W}^{(l)} \mathbf{h}^{(l-1)} + \mathbf{b}^{(l)})$$

donde $\mathbf{W}^{(l)}$ y $\mathbf{b}^{(l)}$ son la matriz de pesos y el vector de sesgos de la capa l .

6.3 Funciones de activación

Nombre	Fórmula	Rango	Uso típico
Sigmoide	$\sigma(z) = 1/(1 + e^{-z})$	$(0, 1)$	Salida binaria
Tanh	$\tanh(z)$	$(-1, 1)$	Capas ocultas
ReLU	$\max(0, z)$	$[0, \infty)$	Capas ocultas (estándar)
Leaky ReLU	$\max(\alpha z, z), \alpha \ll 1$	$\mathbb{R}$	Evita neuronas muertas
Softmax	$e^{z_k} / \sum_j e^{z_j}$	$(0, 1)^K$	Salida multiclase

6.4 Función de pérdida

Para clasificación multiclase se usa la **entropía cruzada categórica**:

$$\mathcal{L} = -\frac{1}{n} \sum_{i=1}^n \sum_{k=1}^K y_{ik} \log \hat{p}_{ik}$$

donde $y_{ik} \in \{0, 1\}$ es el indicador de clase (codificación *one-hot*) y $\hat{p}_{ik}$ es la probabilidad predicha para la clase k .

6.5 Entrenamiento: descenso por gradiente y backpropagation

Los parámetros $\boldsymbol{\theta} = \{\mathbf{W}^{(l)}, \mathbf{b}^{(l)}\}$ se ajustan minimizando $\mathcal{L}$ mediante **descenso por gradiente estocástico** (SGD):

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} \mathcal{L}$$

donde η es la **tasa de aprendizaje** (*learning rate*).

El **algoritmo de backpropagation** calcula eficientemente $\nabla_{\boldsymbol{\theta}} \mathcal{L}$ aplicando la regla de la cadena capa por capa (de la salida hacia la entrada).

Backpropagation no es un algoritmo de optimización por sí mismo: es un método eficiente para calcular el gradiente. El algoritmo de optimización que usa ese gradiente puede ser SGD, Adam, RMSProp, etc.

6.6 Variantes de SGD

- **Mini-batch SGD:** en cada paso se usa un subconjunto (*batch*) de m observaciones (m típicamente entre 32 y 256).
- **Momentum:** acumula gradientes previos para acelerar la convergencia.

- **Adam:** combina momentum con adaptación individual de la tasa de aprendizaje por parámetro; es el optimizador más usado en la práctica.

6.7 Regularización en redes neuronales

- **Dropout:** durante el entrenamiento, cada neurona se desactiva con probabilidad p_{drop} , forzando redundancia en la representación.
- **Regularización L_2 (weight decay):** añade $\lambda \|\boldsymbol{\theta}\|^2$ a la pérdida.
- **Batch normalization:** normaliza las activaciones de cada capa, acelerando el entrenamiento.
- **Early stopping:** detiene el entrenamiento cuando el error de validación deja de decrecer.

7 Métricas de Evaluación para Clasificación

7.1 Matriz de confusión

Para clasificación binaria (positivo/negativo), la **matriz de confusión** organiza las predicciones del modelo:

		Predicho	
		Positivo	Negativo
2*Real	Positivo	VP	FN
	Negativo	FP	VN

donde VP = verdaderos positivos, VN = verdaderos negativos, FP = falsos positivos (error tipo I), FN = falsos negativos (error tipo II).

7.2 Métricas derivadas

$$\text{Exactitud (Accuracy)} = \frac{\text{VP} + \text{VN}}{\text{VP} + \text{FP} + \text{FN} + \text{VN}} \quad (1)$$

$$\text{Precisión (Precision)} = \frac{\text{VP}}{\text{VP} + \text{FP}} \quad (2)$$

$$\text{Sensibilidad (Recall, TPR)} = \frac{\text{VP}}{\text{VP} + \text{FN}} \quad (3)$$

$$\text{Especificidad (TNR)} = \frac{\text{VN}}{\text{VN} + \text{FP}} \quad (4)$$

$$F_1 = 2 \cdot \frac{\text{Precisión} \times \text{Recall}}{\text{Precisión} + \text{Recall}} \quad (5)$$

La exactitud es una métrica **engañosa** con clases desbalanceadas. Si el 95% de los datos son clase negativa, un clasificador que predice siempre negativo tiene accuracy del 95% pero no detecta ningún positivo. En esos casos se prefieren F_1 , AUC-ROC o AUC-PR.

7.3 Curva ROC y AUC

La **curva ROC** (*Receiver Operating Characteristic*) grafica la Sensibilidad (TPR) contra la Tasa de Falsos Positivos ($FPR = 1 - TNR$) para distintos umbrales de clasificación $\tau \in [0, 1]$.

El **AUC** (*Area Under the Curve*) resume la curva en un único valor:

- $AUC = 1$: clasificador perfecto.
- $AUC = 0.5$: clasificador aleatorio (sin poder discriminante).
- $AUC < 0.5$: peor que aleatorio.

AUC-ROC mide la probabilidad de que el modelo asigne un score mayor a una observación positiva aleatoria que a una negativa aleatoria. Es invariante al umbral de decisión y al desbalance de clases.

7.4 Curva Precisión-Recall (PR)

Con datos muy desbalanceados, la curva ROC puede ser optimista. La **curva PR** grafica Precisión vs. Recall para distintos umbrales; el **AP** (*Average Precision*) es el área bajo ella. Es más informativa cuando los positivos son muy escasos.

7.5 Extensión a multiclase

Para $K > 2$ clases:

- **Macro promedio**: promedio no ponderado de la métrica por clase.
- **Weighted promedio**: promedio ponderado por la frecuencia de cada clase.
- **Micro promedio**: agrega VP, FP, FN de todas las clases antes de calcular.

8 Clases Desbalanceadas

En muchos problemas reales (detección de fraude, diagnóstico médico, fallas de hardware) una clase es mucho más frecuente que la otra. Esto perjudica a la mayoría de los clasificadores, que optimizan accuracy global.

8.1 Estrategias de resampling

8.1.1 Oversampling: SMOTE

SMOTE (*Synthetic Minority Oversampling Technique*, Chawla et al. 2002) genera observaciones sintéticas de la clase minoritaria interpolando entre observaciones reales:

1. Para cada observación $\mathbf{x}_i$ de la clase minoritaria, encontrar sus k vecinos más cercanos.
2. Generar un punto sintético $\tilde{\mathbf{x}} = \mathbf{x}_i + \lambda(\mathbf{x}_{nn} - \mathbf{x}_i)$ donde $\mathbf{x}_{nn}$ es un vecino elegido al azar y $\lambda \sim \mathcal{U}(0, 1)$.

8.1.2 Undersampling

Reducir la clase mayoritaria eliminando observaciones al azar o mediante métodos más sofisticados (Tomek links, ENN).

SMOTE debe aplicarse **solo sobre el conjunto de entrenamiento**, nunca sobre el de validación o prueba. Aplicarlo antes de dividir los datos produce data leakage.

8.2 Ajuste de pesos de clase

Muchos clasificadores (RF, SVM, regresión logística) aceptan el parámetro `class_weight` que pondera más los errores en la clase minoritaria durante el entrenamiento:

$$w_k = \frac{n}{K \cdot n_k}$$

donde n_k es el número de observaciones de la clase k .

9 Ingeniería de Features

La **ingeniería de features** (*feature engineering*) es el proceso de transformar los datos crudos en representaciones más informativas para el modelo.

9.1 Transformaciones habituales

- **Estandarización:** $x' = (x - \bar{x})/s$. Obligatoria para SVM y KNN; recomendable para redes neuronales.
- **Normalización min-max:** $x' = (x - x_{\min})/(x_{\max} - x_{\min})$.

- **Transformación logarítmica:** reduce asimetría en variables con larga cola derecha.
- **Codificación de variables categóricas:** one-hot encoding, label encoding, target encoding.
- **Variables de interacción:** $x_1 \cdot x_2$, capturan relaciones no lineales entre pares.
- **Binning (discretización):** convierte variables continuas en intervalos.

9.2 Selección de features

- **Filtrado:** basado en estadísticos univariados (correlación, χ^2 , información mutua).
- **Wrapper:** selección basada en el rendimiento del modelo (RFE – *Recursive Feature Elimination*).
- **Embedded:** regularización LASSO, importancia en Random Forest.

10 Comparación de Modelos: Grid Search y Pipelines

10.1 Búsqueda de hiperparámetros

Los hiperparámetros no se aprenden durante el entrenamiento; deben fijarse externamente. Las estrategias de búsqueda son:

- **Grid search:** evalúa todas las combinaciones de una grilla prefijada de valores.
- **Random search:** muestrea combinaciones al azar de distribuciones de valores; más eficiente que grid search para espacios grandes.
- **Búsqueda bayesiana:** utiliza un modelo probabilístico del rendimiento para guiar la búsqueda.

Observación 10.1. La evaluación durante la búsqueda de hiperparámetros debe realizarse con validación cruzada **sobre el conjunto de entrenamiento**. El conjunto de prueba queda reservado para la evaluación final del modelo seleccionado.

10.2 Pipelines

Un **pipeline** encadena pasos de preprocesamiento y modelado en un único objeto, garantizando que:

1. Las transformaciones se aprenden solo sobre el fold de entrenamiento en cada iteración de CV.

2. No hay filtración de información del conjunto de validación.
3. El código de producción es reproducible y no presenta discrepancias respecto al entrenamiento.

El uso de pipelines es una práctica esencial de ML responsable: evita el **data leakage** y simplifica el despliegue del modelo.

Resumen de la Unidad

Modelo	Supuesto clave	Ventaja	Limitación
Árbol de decisión	Sin supuestos distribucionales	Interpretable	Alta varianza
Random Forest	IID, suficientes features	Robusto, importancia	Caja negra
SVM	Separabilidad (kernel)	Eficaz en alta dim.	Costoso en n grande
KNN	Proximidad implica similitud	Sin entrenamiento	Maldición dim.
MLP	Aproximador universal	Muy flexible	Requiere muchos datos

Referencias

- Breiman, L. (2001). *Random Forests*. Machine Learning, 45(1), 5–32.
- Chawla, N. V., et al. (2002). *SMOTE: Synthetic Minority Over-sampling Technique*. JAIR, 16, 321–357.
- Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (3rd ed.). O'Reilly Media.
- Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning* (2nd ed.). Springer.
- James, G., et al. (2021). *An Introduction to Statistical Learning with Applications in R* (2nd ed.). Springer.